

RESEARCH

Open Access



LogLAA: an adaptive integrated log anomaly analysis framework

Yali Gao^{1*†} , Tianchao Luo^{1†}, Kangqian Huang^{1,2}, Jialu Tang¹ and Xiaoyong Li¹

Abstract

Modern systems generate massive amounts of logs during operation, which are the key foundation for anomaly log analysis. However, existing research typically breaks down log analysis into multiple isolated tasks, which lacks flexibility in complex application scenarios and requires significant manpower. Furthermore, the increasing diversity and complexity of log formats place higher demands on the accuracy of log analysis. To achieve a more robust, accurate, and comprehensive log analysis method, we propose an integrated framework, called LogLAA. We construct a log parser based on length and word frequency that runs stably in most log systems with minimal parameter tuning, supporting both offline and online parsing in various scenarios. By introducing variable substitution and combining it with a similarity prefix tree, we achieve high accuracy and efficiency. We introduce counting embeddings, sequence embeddings, and semantic embeddings, and combining them with a CNN-LSTM model based on a dual-attention mechanism, we significantly improve the precision of anomaly detection. To ensure the interpretability of anomaly logs, we combine them with a large language model (LLM) for analysis. Experimental results show that our log parsing method achieves a 0.8% improvement over the SOTA model and anomaly detection achieves 6% improvement over the average precision of other advanced methods. We use the weighted matching score to evaluate anomaly analysis. LogLAA scores 0.7, placing it at an upper-middle level.

Keywords Log parsing, Anomaly detection, Anomaly log analysis

Introduction

With the rapid development of computer technology, systems are gradually evolving towards large-scale, and system throughput has also been significantly improved. Software systems are becoming increasingly large and complex and are subject to more failures. Large-scale distributed services are usually composed of many nodes and support millions of users at the same time. Anomalies in the system may cause the program to crash,

seriously affecting the availability and reliability of the distributed system. These anomalies include network interruptions, blocking, and others. The logs generated by the system record the detailed status and key events of the system operation. Anomaly detection has become an important research topic that has attracted much attention (He et al. 2025; Guan et al. 2024; Zhao et al. 2025; Meng et al. 2019; Zhang et al. 2019; Ying et al. 2021). In a distributed environment, as the scale of the system continues to grow, the amount of logs generated also increases significantly (Ying et al. 2021), which greatly increases the complexity of log anomaly detection.

In existing research, the tasks of log analysis are divided into discrete sub-tasks (Liu et al. 2025). The most widely studied areas are log parsing and log anomaly detection. Log parsing converts messy logs into a structured form, while log anomaly detection identifies whether logs are abnormal. Prediction based solely on anomaly detection

[†]Y. Gao and T. Luo these authors contributed equally to this work.

*Correspondence:

Yali Gao

gaoyali@bupt.edu.cn

¹ Key Laboratory of Trustworthy Distributed Computing and Service (BUPT), Ministry of Education, Beijing University of Posts and Telecommunications, Beijing 100876, China

² Guangdong Power Trading Center Co., Ltd, ITD, Guangzhou, Guangdong 511510, China

seems inadequate for meeting the needs of high-efficiency industries, so log interpretation (Liu et al. 2024) has become a new research topic. The purpose of log interpretation is to analyze the main cause of the anomaly and propose repair suggestions. Discrete log analysis sub-tasks often yield good results in single and simple scenarios, but the increasing diversity and complexity of system logs make log analysis more challenging. The lack of a unified log analysis framework also adds a lot of difficulty to the analysis process. We must solve the following challenges to achieve robust, efficient, and complete log analysis in distributed scenarios.

- *Logs formats change rapidly, and most methods cannot adaptively parse multiple types of logs.* In actively developed and maintained software systems, log formats often change, accounting for about 20-45% of the total logs (Zhang et al. 2019). Therefore, it is crucial to introduce new log parsers or similar methods to handle changes in log formats, which ensures that anomaly detection methods can adapt to different log formats.
- *Single feature extraction, unstable performance, and immature anomaly analysis.* Current log anomaly detection methods typically focus on log sequence anomalies while neglecting log quantity anomalies and semantic anomalies (Landauer et al. 2023; Le and Zhang 2022). Most methods also ignore the post-detection analysis process. Current research mainly focuses on solving the log anomaly discovery problem, while neglecting the field of automated anomaly analysis. However, the subsequent discovery and resolution of system anomalies are equally important.
- *There is no integrated log analysis framework, in which tasks are fragmented and data processing is difficult.* The current log analysis tasks include discrete sub-tasks. Each task has its own data set and benchmark. Therefore, for different sub-tasks, the data format and content need to be constantly adapted. Logs with complex formats are particularly difficult to process. Without an integrated analysis framework, the overall workload of the analysis will be excessively large.

To address the aforementioned issues, this paper introduces LogLAA, an integrated log analysis framework, which includes two main stages: offline and online. Each stage contains four key steps: log parsing based on length, word frequency and combinatorial similarity clustering, multi-feature extraction and fusion, log anomaly detection based on the dual-attention mechanism CNN-LSTM, anomaly analysis based on large language models (LLMs).

In the offline phase, LogLAA preprocesses data based on different components and analyzes the correlation between logs and IDs. A major innovation of LogLAA is the introduction of a log parser based on word frequency and word length in the log parsing stage. This parser extracts log templates with minimal parameter tuning, thus ensuring the semantic integrity of sentences. LogLAA supports offline log template extraction and online clustering template dynamic matching by constructing a prefix tree. LogLAA also introduces a hybrid similarity calculation mechanism to ensure the accuracy of template extraction. LogLAA integrates semantic information into high-dimensional vector embeddings through projection and extracts quantitative information with a sliding window. In the anomaly detection stage, LogLAA uses a CNN-LSTM model based on a dual attention mechanism to learn fixed patterns in the log data. The attention mechanism enhances the model's robustness to small fluctuations and increases the weight of key features. In the online phase, LogLAA includes four tasks: (1) performing log parsing and anomaly prediction on the online input logs; (2) mapping logs according to log IDs and summarizing the predicted log data; (3) deriving the predicted anomaly system logs; and (4) analyzing and interpreting the causes of anomalies using LLMs. When the anomaly logs are input into LogLAA, it sequentially performs the four tasks mentioned above. Finally, we validated LogLAA's performance on widely used large-scale datasets. We summarize the contributions of the proposed algorithm:

- *We have implemented an integrated log analysis framework that covers the entire process of log parsing, anomaly detection, and anomaly analysis without any intermediate processing.* Our LogLAA framework has been aggregated each discrete log analysis sub-task into a complete analysis framework. It is capable of handling the entire process from raw logs to anomaly detection and analysis without any manual intervention. Our method has been evaluated on mainstream open-source datasets and its feasibility have been verified through comparative and ablation experiments.
- *We have implemented an adaptive log parser and log analysis method that can adapt to various formats of log data.* This parser requires minimal configuration adjustments to adapt to various log formats, enhancing the stability and robustness of log anomaly detection in distributed systems. We introduce multi-feature embeddings, combined with a CNN-LSTM model and a dual attention mechanism, significantly improving anomaly detection precision. Log analysis

can directly obtain the model's predicted output and perform analysis using preset instructions.

- *We conducted a comprehensive evaluation of LogLAA, covering all the tasks involved.* The evaluation results highlight the efficiency and effectiveness of the framework, validating its excellent performance.

The rest of this paper is organized as follows: Section II introduces the relevant process of log anomaly detection. Section III further discusses our framework, including log parsing, anomaly detection, and anomaly analysis. Sections IV and V show the evaluation results and related discussions of the proposed model.

Related work

Log parsing

With the rapid development of computer technology, the scale of system logs continues to expand. The structural characteristics of logs are influenced by factors such as generation time and system type, which prevents logs from being fully structured. Therefore, it is crucial to parse unstructured log text into structured representations. Common log parsing methods are represented by Spell (Du and Li 2016), which is an online unsupervised streaming parser based on the LCS (Longest Common Subsequence) idea. Most logs are valid words that can be found in dictionaries, and log messages with the same log event are likely to have similar lengths. Logram (Dai et al. 2020) uses frequent pattern mining to identify variables and constants, finding frequently occurring n -gram patterns in the data set as templates. At the same time, heuristic-based log parsing methods also show high accuracy. For example, the AEL (Jiang et al. 2008) method assumes that log messages generated by the same event have textual similarities and classifies logs through anonymization and tokenization. The method does not require users to set parameters in advance, but requires manual intervention to formulate rules for merging event types. The Drain algorithm (He et al. 2017) relies on a fixed-depth tree structure for log analysis. Therefore, we propose LogLAA, a integrated log analysis framework includes a log parsing method based on word frequency, length, and similarity clustering. This method requires only minor parameter tuning and can guarantee high log parsing accuracy when using different log formats.

Log anomaly detection

Log vectorization is the process of converting textual log data into vector representations. In log analysis, common methods of log vectorization include CBOW (Kenter et al. 2016), TF-IDF, Word2Vec, BERT (Devlin et al. 2019), and ALBERT (Lan et al. 2019). CBOW represents text as binary values or frequency counts for words

within a vocabulary. However, it overlooks word order and semantic context. TF-IDF measures word importance based on term frequency and inverse document frequency. Nonetheless, TF-IDF relies on a pre-existing vocabulary and fails to provide effective vector representations for new words not present in the vocabulary. Log sequence anomalies refer to the irregularities in the execution order of logs within a log sequence (Kim et al. 2021). Typically, programs follow fixed processes, and process entities continuously generate log records of their behavior in chronological order during runtime. If the log sequence deviates from the normal flow of the program, it is termed as a log sequence anomaly. Several researchs have focused on detecting such anomalies (Zuo et al. 2022). For instance, DeepLog employs LSTM to predict the next log likely to appear after observing a log. An anomaly is declared if the actual log deviates from the prediction. LogAnomaly extracts log sequences using a sliding window and combines LSTM for anomaly detection (Meng et al. 2019). LightLog replaces LSTM with TCN, significantly reducing the parameter count and enabling sequence anomaly detection (Wang et al. 2022). The semantic features of logs focus on inconsistencies or irrationalities in events at the semantic level, which are often closely related to abnormal system behavior, potential failures, or safety hazards (Sun et al. 2023). In such anomalies, the structure and order of log entries often appear normal, or even completely conform to the log template or execution process. However, some log content appears contradictory, illogical, or deviates from expected business behavior in specific context, thus constituting anomalies at the semantic level. In order to detect such anomalies, common methods involve using technologies such as deep learning and pre-trained language models to deeply explore semantic patterns and anomalies in log data, enabling efficient context modeling and deep semantic understanding. LogRobust (Zhang et al. 2019) uses the pre-trained FastText model to calculate the semantic vector of log events. LogAnomaly (Meng et al. 2019) obtains semantic vectors from language models. Log quantity anomalies refer to the disruption of constant linear relationships during program execution, often captured through log quantity relations. To detect such anomalies, a common approach is to define a time or session window and use the count of each template index within the window as the basis for anomaly detection. Supervised machine learning methods, such as logistic regression can be employed for anomaly detection. Unsupervised methods like PCA (Xu et al. 2009) and LogCluster (Vaarandi and Pihelgas 2015) can also be utilized for detecting quantity anomalies. These diverse methods offer various approaches to detecting anomalies in a system, contributing to improved system

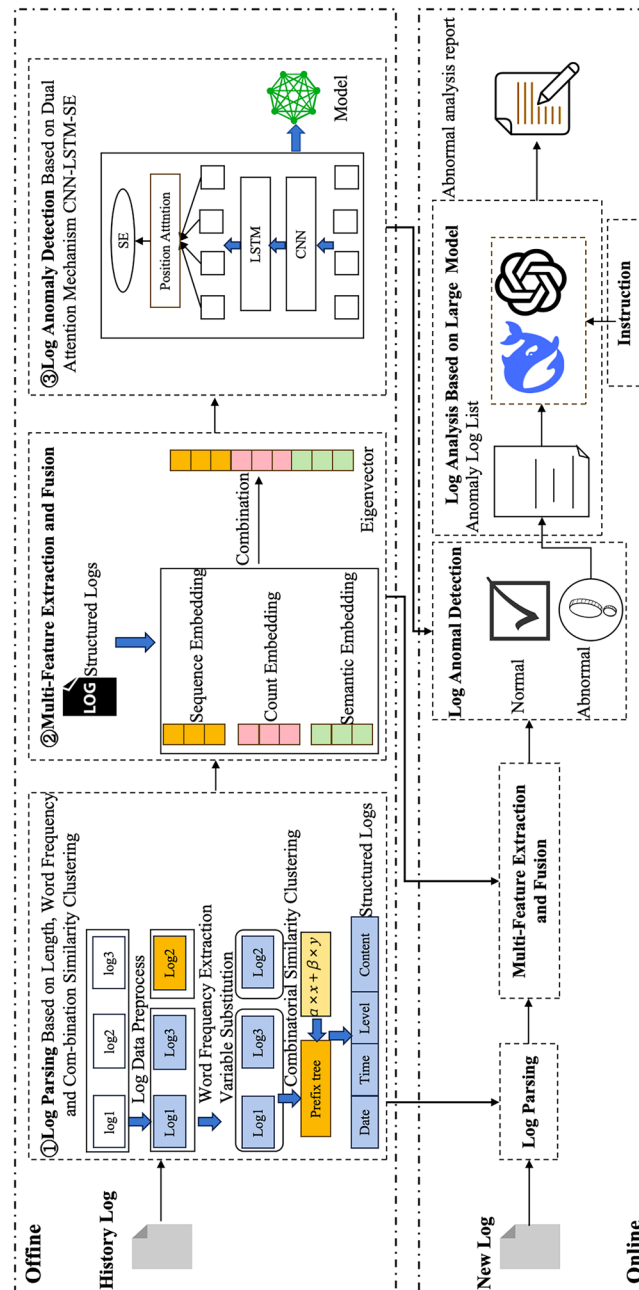


Fig. 1 LogLAA Overview. LogLAA comprises two main stages: offline processing and online processing. The offline processing stage includes log parsing based on length, word frequency, and combinatorial similarity clustering, multi-feature extraction and fusion, and log anomaly detection based on the dual-attention mechanism CNN-LSTM. The online processing stage includes log parsing, multi-feature extraction and fusion, log anomaly detection, and log analysis based on LLMs

reliability and efficiency. In recent years, deep learning based methods have performed well in log anomaly detection tasks. These methods convert unstructured logs into structured sequences through log parsing and use deep learning models to discover the features in these log sequences. Neural network models commonly used for anomaly detection include RNN, GRU, LSTM, and CNN. However, RNN networks need to calculate the states of neurons step by step, so they are not suitable for parallel processing and are susceptible to the vanishing gradient problem (Duan et al. 2025). Since LSTM networks perform well in time series anomaly detection and overcome the shortcomings of RNN networks, they are gradually being used in log anomaly detection (Novaes et al. 2020). GRU combines the input and forget gate of LSTM to simplify the complexity of the model. In order to better improve the detection effect, LogLAA integrates different types of features to obtain a more comprehensive perspective. We introduce sequence information mapped by the word mapping table to detect logical anomalies. By counting the number of times each template appears in the sliding window, we extract quantitative features, and analyze the number of events. Based on the sequence features, quantitative features, and semantic features obtained above, they are normalized to obtain the feature vector corresponding to the log as the model

input. CNN-LSTM-SE is a hybrid deep learning model that integrates convolutional neural network (CNN), long short-term memory network (LSTM), and Squeeze-and-Excitation (SE) modules. It is widely used in the analysis and processing of time series data. It uses the advantages of these three models to improve the ability to learn complex patterns in time series data, especially in tasks such as log analysis and anomaly detection. Therefore, we input the combined features into the CNN-LSTM model based on the attention mechanism and SE module to significantly improve the detection precision.

Log anomaly analysis

Log anomaly analysis is to accurately obtain the cause and solution of the anomaly. In recent years, a large number of related researchs have focused on log parsing and anomaly detection, but there is a lack of anomaly analysis. Based on the powerful comprehension and thinking abilities of LLMs, they can handle large amounts of data input and perform causal analysis to improve efficiency (Xiao et al. 2024). The RCACOPLOT (Chen et al. 2024) system uses fine-tuning LLMs for the analysis of cloud computing events and has achieved effective results. This shows that LLMs have obvious advantages in cause analysis. Log anomaly analysis is often treated as a “black box”, although anomalies can be detected, their underlying causes are not explicitly revealed. Therefore, LogLAA leverages LLMs to analyze anomalous logs and obtain corresponding solutions.

Our method

Figure 1 outlines our approach to building LogLAA. The entire process can be divided into two parts: an offline part and an online part. The symbols used in this paper and their descriptions are shown in Table 1.

Log parsing based on length, word frequency and combination similarity clustering

Log parsing has been extensively studied over the years as a crucial stage before performing downstream log analysis tasks. Previous research often involves the introduction of many parameters, such as message type thresholds in Spell (Du and Li 2016), similarity thresholds in Drain (He et al. 2017). Our goal is to propose a log parsing method that can adapt to most common logging systems using minimum parameter tuning. Current log parsers usually introduce a large number of parameters that need to be determined after multiple tests. Logs usually present system events in natural language, most of which are valid words whose definitions can be found in dictionaries. In addition, through Drain’s (He et al. 2017) experiments, it was found that log messages with the same log event may have similar lengths and

Table 1 NOTATIONS AND DESCRIPTIONS

Notations	Description
α	Coefficients or weights
$freq$	Frequency of occurrence
i	Position index
A, B	Sets of token
w_i	Position weight
$S_{jaccard}$	Jaccard similarity
V_{w_i}	The word vector of the i -th word
$E_{Artificial}$	Manually labeled feature vectors
E_{Model}	Feature vector generated by the model
$Threshold$	Root node selection threshold
G	Current image
H	Current feature map
H_{se}	Weighted feature map
s	Channel attention coefficient
c	Input features
$\cos$	Cosine similarity
$\mathcal{L}$	Loss function value
b	Output of LSTM
d	Output of SE module
e	The result after average pooling
V_{result}	Context vector obtained by attention mechanism
$average_pooling$	Average Pooling
$compute_loss$	Calculating the loss function

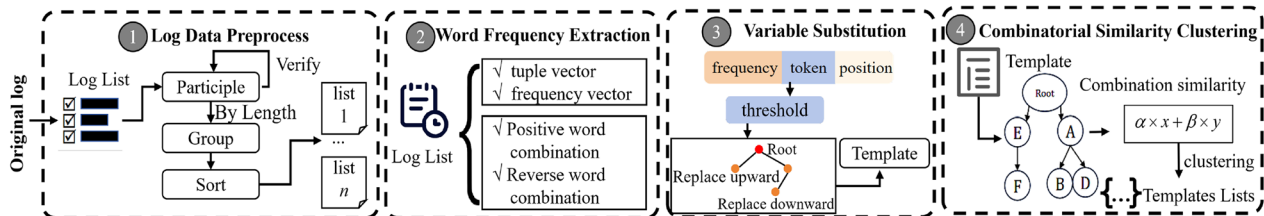


Fig. 2 Log Parsing Based on Length, Word Frequency and Combination Similarity Clustering workflow. The log parser mainly consists of four components: Log data preprocess, word frequency extraction, variable substitution, and similarity clustering

multiple repeated keywords. We proposed a method based on word frequency and length, as shown in Fig. 2. The log parser regards log messages within a specified similarity threshold and the same log length as templates. This approach requires minimal parameter tuning while achieving high accuracy and efficiency. Our main hyperparameters and default values are set as follows: the maximum depth of the prefix tree matching is set to 4, the variable identification threshold is set to 2, the sequence similarity threshold is fixed at 0.4, and the maximum number of child nodes for each prefix tree node is 100. For sequence matching, we use a fixed combined similarity design, which is deterministic and does not introduce any other adjustable hyperparameters except for the similarity threshold.

Log data preprocess

We will preprocess the incoming log entries. The system implements standardization on each log. Specifically, for dynamic information that may be contained in the log (such as IP addresses, numbers, timestamps, etc.), the system uses predefined regular expressions to match them one by one and replaces the matched parts with the wildcard "<*>". The purpose is to remove noise in the log so that subsequent processing can focus more on the structure of the log rather than the specific values. After the replacement is completed, the system uses preset delimiters (such as commas, periods, semicolons, etc.) to split the processed log into independent tokens. Then, we group the logs according to the number of tokens and build a structure that records the number of tokens in each group of logs. The length is used as the basis for this round of grouping.

Word frequency

After length grouping, the system uses a temporary dictionary to count the frequency of each token at each position across all logs. Iterate through the positions of each token in all logs, storing the token and its position as keys in the dictionary. If a token has already appeared

at a specific position, its frequency is incremented; otherwise, the count is initialized. In this way, a global frequency can be obtained for each position and token combination. Since the first token of each log is the line number, this item will be skipped in subsequent statistics. For each token at position i , retrieve its frequency at position $i+1$ based on prior statistics. For each token, construct a triple: (frequency, token, position). The triples corresponding to all tokens in a log are formed into a list. This list forms the "tuple vector" of the log, while a "word frequency vector" is also generated. This vector only stores the frequency value of the token at each position. These two vectors are stored in the corresponding dictionaries, and their keys are the number of tokens in the log.

Variable substitution

After preprocessing and word frequency extraction, three data structures are obtained by grouping logs according to their length:

- **group_len**: the token list of the same group of logs (grouped by log length);
- **tuple_vector**: the list of tuples corresponding to each log, each tuple contains (frequency, token, position);
- **frequency_vector**: the vector of each log consisting only of token frequencies.

For each log in each group, the system sorts its tuple list in descending order based on the frequency value of each tuple. At the same time, two word combinations are generated: forward word combination and reverse word combination for subsequent template extraction reference. Then, the context adjustment method is used to perform preliminary extraction of candidate templates. The word combinations in each group are traversed. First, the token with the highest frequency in each combination is determined, and the threshold is calculated. Each token is checked in turn. If its frequency reaches or exceeds the threshold, it is considered that the token is stable enough to be used as the "root" token of the

template; otherwise, the candidate with the highest frequency is selected, and the consistency of the candidate token in its upper position is checked. The dynamic root node selection threshold is defined as follows:

$$\text{Threshold}(G) = \alpha \times \max_{w \in G} (\text{freq}(w)), \quad (1)$$

where α is the threshold ratio parameter, $f(w)$ is the frequency of occurrence of word w . Where $\text{Threshold}(G)$ is the dynamic root node selection threshold of the graph, G is the current graph, α is the adjustment parameter which is a pre-set coefficient, and $\text{freq}(w)$ is the frequency of occurrence of node w , and $\max$ indicates the maximum value.

If the parent node part is unstable or mismatched, these positions are uniformly replaced with wildcards "<*>". The distribution of candidate tokens in the lower layer is checked. For those tokens with large changes, they are replaced with "<*>" again to further eliminate the noise of the variable part. Finally, a more generalized and stable log template is obtained after upper and lower layer adjustments. Through length grouping, multi-dimensional frequency vector extraction, preset threshold ratios, and variable substitution, the system ensures the selection of high-frequency and stable tokens. Additionally, upper and lower layer optimization meets the needs of dynamic changes in the log.

Combinatorial similarity clustering

After variable substitution, we can get the log templates. Then we build the prefix tree structure and set the pre-processing module before inserting the prefix tree. This can not only directly accept the template generated after the variable substitution step for clustering offline, but also support online template input and parsing matching.

For each template generated in the offline stage (for example, "PacketResponder <*> for block <*> terminating"), insert it into the prefix tree in token order, using the log length as the keyword to ensure that log templates of different lengths are stored separately. If the token already exists in the child node of the current node, continue along the branch. If not, create a new node and insert the token into the child node dictionary. When encountering a token containing a number or a large change, you can choose to use a wildcard node ("<*>") to reduce template bifurcation and avoid overly detailed distinctions. Reasonable parameter settings will limit the number of prefix tree nodes to prevent over-parsing. For the input log sequence, the corresponding first-layer node is found according to the number of log tokens. The leaf nodes reached during the search process constitute the candidate cluster set. For each template in the candidate cluster, the similarity is calculated using the combined weight similarity method. The position weight and Jaccard similarity are combined to evaluate the degree of match between the two. If the similarity exceeds the preset threshold, the match is considered successful. If a matching cluster is found, the log is classified into the cluster. If no match is successful, a new object is created to represent the new cluster and inserted into the prefix tree.

In order to enhance the influence of the tokens at the front of the sequence, the following position weight definition is introduced:

$$w_i = \frac{1}{i + 1}, \quad (2)$$

where w_i is the weight of the i -th position in the sequence, and i is the position index.

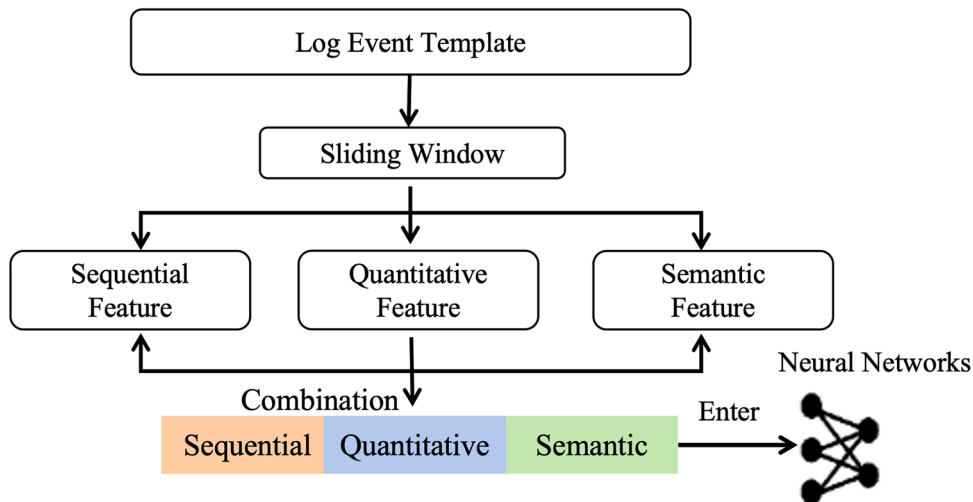


Fig. 3 Sentence Embedding Workflow

The non-wildcard ($<*>$) tokens in the two sequences are grouped into sets, and their Jaccard similarity is:

$$S_{\text{jaccard}} = \frac{|A \cap B|}{|A \cup B|}, \quad (3)$$

where S_{jaccard} is the Jaccard similarity between set A and set B , A is the set of all non-wildcard tokens in the first sequence and B is the set of all non-wildcard tokens in the second sequence.

The final matching similarity combines position weighted matching and Jaccard similarity, and is defined as follows:

$$S_{\text{combined}} = \alpha \cdot w_i + (1 - \alpha) \cdot S_{\text{jaccard}}. \quad (4)$$

Multimodal feature extraction and fusion

After passing through the log parser, the corresponding log template is obtained. This section processes the log template, converting it into three modalities containing semantic, sequence, and quantity information, as shown in Fig. 3. We overcome the instability caused by log changes by obtaining log semantic information and mapping it to vectorization to detect performance issues. Finally, we use a sliding window to obtain count information to detect the number of abnormal logs.

Semantic embedding

Although the log format is constantly changing, the key meaning of the log statement remains unchanged (Zhou et al. 2022). In the log anomaly detection system, each log contains an event template, whose semantic information is crucial for detecting semantic anomalies. This paper adopts the weighted average aggregation of the pre-trained FastText word vector model to generate a fixed-dimensional template semantic vector. First, the original template is standardized to obtain a word sequence. The vector representation of each word is obtained through the pre-trained word vector model. Finally, the semantic embedding vector of the template is calculated by the weighted average function.

$$\mathbf{v}_{\text{sem}} = \frac{1}{n} \sum_{i=1}^n \alpha_i \mathbf{v}_{w_i}, \quad (5)$$

where α_i denotes the weight coefficient that corresponds to the i -th word vector, and $\mathbf{v}_{w_i}$ is the word vector that corresponds to the i -th word.

Sequence embedding

Sequence features are used to capture the sequential dependencies between log events and reflect the temporal logic of program execution. A normal system should follow a certain execution process, and the temporal logic should be clear and stable. When this balance is broken, it may be the cause of the exception. The extraction of sequence features is to map the log template to an integer ID to form a fixed-length sequence.

Count embedding

In addition to sequential patterns, template vector sequences also contain quantitative patterns. Normal program execution usually follows certain rules. For example, each open file will eventually be closed. Therefore, under normal circumstances, the number of “open file” logs should be equal to the number of “close file” logs. These quantitative relationships may reflect normal program execution behavior. When new logs break these relationships, we can judge that the execution of the system is abnormal. The counting feature extracts the frequency of each event template in the sliding window. First, each event template is identified from the log data, and the number of times each event appears in the log is counted. For example, in a set of user activity logs, if the “user login” event appears 5 times and the “user performs an operation” event appears 3 times, the frequency of occurrence of these two events can be recorded. Then, this frequency information is combined with the original log data so that each log record contains the number of occurrences of its corresponding event. Providing rich feature information for subsequent analysis and model training.

CNN-LSTM based on dual attention mechanism**Algorithm 1** CNN-LSTM With Dual Attention

Input: $\mathbf{V} = [V_1, V_2, \dots, V_w]$
Output: $\mathcal{L}$

```

1: model  $\leftarrow$  CNNLSTMSE(config)
2: forward(batch, device)
3:  $x \leftarrow \text{concat}(\text{extract\_features}(\text{batch}, \mathbf{V}), \text{dim} = -1)$ 
4:  $x \leftarrow \text{CNN}(x)$ 
5:  $b \leftarrow \text{LSTM}(x)$ 
6:  $a \leftarrow \text{softmax}\left(\frac{1}{H} \sum_{d=1}^H H_{t,d}\right)$  //see Eq. (6)
7:  $c \leftarrow b \odot a$ 
8:  $z \leftarrow \text{ReLU}(W_1 c)$  //see Eq. (7)
9:  $s \leftarrow \sigma(W_2 z)$  //see Eq. (7)
10:  $H_{sc} \leftarrow b \otimes s$  //see Eq. (8)
11:  $e \leftarrow \text{average\_pooling}(d)$ 
12:  $\text{logits} \leftarrow \text{fc}(e)$ 
13: if label_in_batch then
14:    $\mathcal{L} \leftarrow \text{compute\_loss}(\text{logits}, \text{label})$ 
15: end if
16: return logits,  $\mathcal{L}$ 

```

The vectors (like $\mathbf{V} = [V_1, V_2, \dots, V_w]$) of the log sequence are used as the model input. The CNN-LSTM-SE neural network performs multi-dimensional processing on the logarithmic sequence, extracting local features through convolution of the CNN layer and processing the long-range dependencies of the sequence data through the LSTM network. The pseudocode is shown in Algorithm 1. To make the model pay more attention to the more valuable information in the logarithmic sequence, this paper uses the context vector generated by the positional attention mechanism to replace the traditional global average pooling. As shown in Fig. 4, this method not only compresses the sequence length but also retains key temporal information. To enable the model to dynamically calibrate the importance of feature channels, this paper introduces the SE channel attention module to calculate the feature channel weights. The position attention mechanism weights are defined as follows:

$$a = \text{softmax}\left(\frac{1}{H} \sum_{d=1}^H H_{t,d}\right), \quad (6)$$

where $H_{t,d}$ is the feature response at position t and channel d , and H is the total number of channels in the feature map.

The SE channel attention mechanism is divided into compression and excitation and feature recalibration, which are defined as follows:

$$z = \text{ReLU}(W_1 c), \quad s = \sigma(W_2 z), \quad (7)$$

where z is the intermediate feature obtained by transforming feature c through the ReLU activation function, W_1 is the weight matrix related to the input feature c , c is the input feature, σ is the Sigmoid activation function, and W_2 is the second weight matrix.

$$H_{se} = H \otimes s, \quad (8)$$

where H_{se} is the feature map after channel attention weighting, $\otimes$ is the Hadamard product, H is the original feature map, and s is the channel attention coefficient.

After global pooling and full connection, the final prediction output is determined by applying a Softmax layer to this summarized hidden state vector. The calculation process can be expressed as

$$\text{pred} = \text{softmax}\left(W' \left(\sum_{i=0}^w \alpha_i h_i\right)\right), \quad (9)$$

where W' stands for the weight of the *softmax*() layer.

Anomaly analysis based on large language model

LLMs (such as ChatGPT) possess powerful language understanding and generation capabilities (Duan et al. 2025; He et al. 2024). They can conduct indepth analysis of anomaly logs after anomaly detection. By combining anomaly logs with relevant instructions, the model can leverage its rich knowledge base and contextual understanding to infer potential causes of anomalies.

Algorithm 2 Anomaly Analysis Based on LLM

Input: y_pred , $idxs$, idx_to_log
Output: analysis results

```

1:  $log\_groups \leftarrow \{\}$ 
2:  $create\_dir(outputs)$ 
3:  $output\_file \leftarrow outputs/predicted.txt$ 
4:  $anomaly\_count \leftarrow 0$ 
5: for all  $(y_i, idx) \in y\_pred$  do
6:   if  $y_i = 1$  then
7:      $log\_text \leftarrow idx\_to\_log[idx]$ 
8:     if  $log\_text = None$  then
9:        $log\_text \leftarrow \text{Missing log}$ 
10:    end if
11:     $append(output\_file, log\_text)$ 
12:     $anomaly\_count \leftarrow anomaly\_count + 1$ 
13:  end if
14: end for
15: if  $anomaly\_count > 0$  then
16:    $print(\text{"anomaly logs have been saved"})$ 
17: else
18:    $print(\text{"no anomalies predicted"})$ 
19: end if
20: if  $is\_empty(idx\_to\_log)$  then
21:    $print(\text{"log mapping is empty"})$ 
22: end if
23:  $send\_to\_llm(outputs/predicted.txt, instruction)$ 
24: return analysis results

```

However, directly feeding all logs into the model for analysis lacks effective contextual filtering and targeted analysis, potentially leading to misinterpretations or inaccurate analysis results. Furthermore, processing large amounts of log data consumes significant computing resources. Therefore, systematic log parsing and format standardization are crucial. Through anomaly detection and log extraction, we can generate an accurate list of anomaly logs, effectively reducing computational costs and optimizing model analysis. By combining the understanding and reasoning capabilities of large models with refined log processing, more accurate anomaly cause analysis and more effective anomaly diagnosis guidance can be achieved.

After the training phase, the prediction model is applied to offline logs. When online logs are received, they are first processed by the log parsing and anomaly detection modules. LogLAA then organizes and summarizes the logs predicted to be anomalous. It uses a log ID mapping approach based on the structured log directory produced by the log parser, which contains two columns: log ID and log event. The log ID serves as the key, and the event as the corresponding value.

Once the model completes prediction, a dictionary-format result is generated, where the key corresponds

to the log index (i.e., log ID), and the value indicates the predicted anomaly status. Based on this mapping, LogLAA will extract the logs that are predicted to be abnormal and export them for aggregation. The pseudocode is shown in Algorithm 2.

Furthermore, the LogLAA framework integrates a log analysis module. To obtain more accurate and efficient anomaly analysis results, LogLAA incorporates predefined analysis prompts and connects to APIs of LLMs (such as DeepSeek). Logs identified as anomalous, along with the fixed instructions, are sent to the language model, which generates corresponding anomaly analysis reports. This design enhances the completeness of the log analysis framework and lays a solid foundation for automated anomaly diagnosis. The models and prompts used for anomaly analysis and evaluation are summarized in Fig. 5.

Experiment and analysis

DataSet

LogPAI is a log parsing benchmark that uses 16 real-world log datasets covering distributed systems, supercomputers, operating systems, mobile systems, server

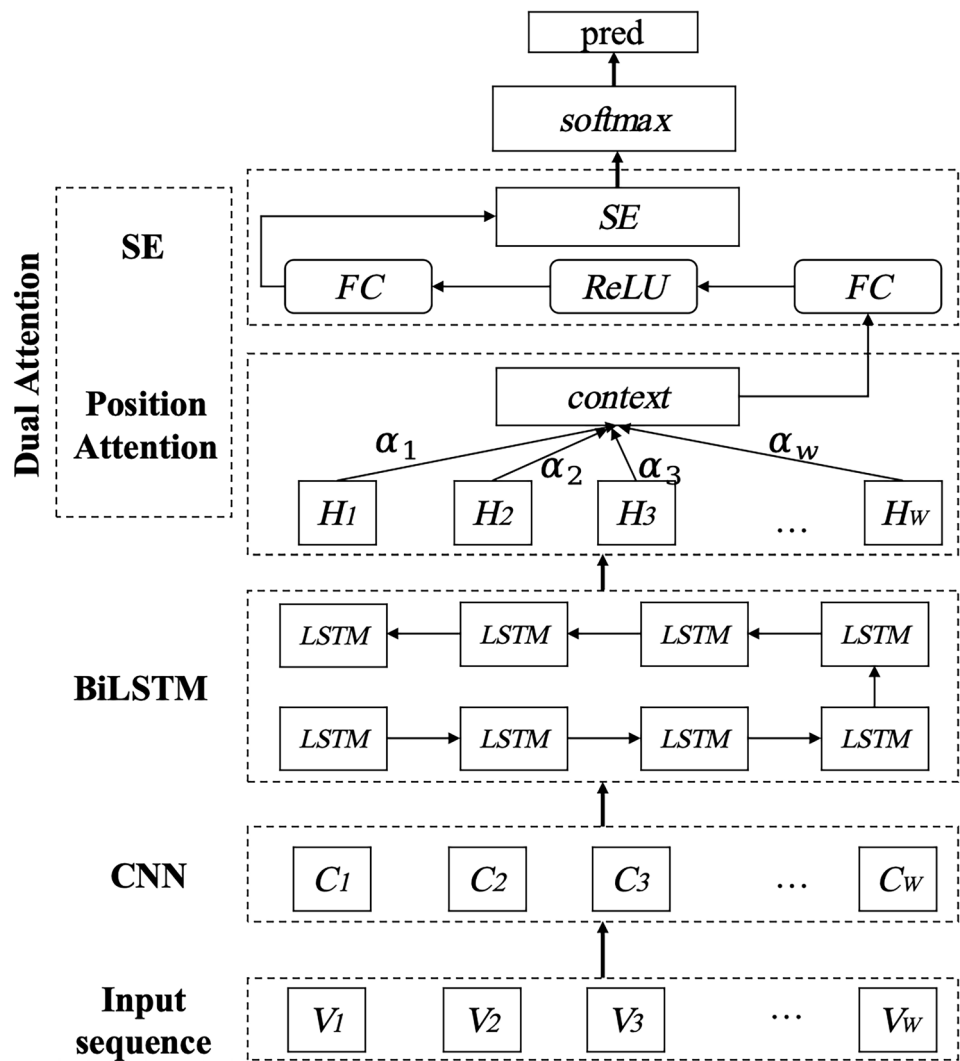


Fig. 4 CNN-LSTM architecture based on dual Attn

applications, and standalone software. HDFS (Hadoop Distributed File System), Hadoop, BGL (Blue Gene/L Supercomputer Log), and HPC (High Performance Computing) (Zhu et al. 2019) are provided by LogHub (Zhu

et al. 2023). The source data in the log parsing part is the Loghub2k dataset, which contains 2,000 real log data and templates from various fields. For log anomaly detection, we selected two sets of datasets from LogHub for performance testing. The overall description of log datasets is shown in Table 2. The HDFS log data set is composed of log data from HDFS. HDFS is an open source distributed

Anomaly Analysis Model: Deepseek V3
Anomaly Analysis Test Model: ChatGPT 4

Prompt:
You are a senior system security engineer. Please analyze the following system exception log:
Requirements:
1. Exception type;
2. Possible causes;
3. Suggested troubleshooting and repair methods;
4. Affected modules or components (if any).

Fig. 5 Anomaly Analysis Models and Instructions

Table 2 Dataset Overview

Dataset	Dataset Size	Number of Logs
HDFS (1)	282KB	2000
HDFS (2)	1.55G	11,175,629
Hadoop	376KB	2000
BGL (1)	308KB	2000
BGL (2)	708M	4,747,963

file system that is widely used for large-scale data storage and processing. The HDFS dataset is generated by running MapReduce jobs on more than 200 server nodes and has been manually labeled. This data set includes various events and error information in the HDFS cluster, such as NameNode startup, shutdown events, DataNode join and exit cluster events (Chnib and Gabsi 2023). This dataset can be used to train and evaluate various log anomaly detection algorithms to help improve system reliability and security. The BGL data set is another widely used log anomaly detection data set. This dataset was produced by researchers at McGill University in Canada to evaluate the performance of a machine learning-based log anomaly detection system they developed. The BGL dataset is extracted from the running logs of the BlueGene/L supercomputer produced by IBM and is labeled with normal and abnormal labels. The BGL dataset includes log files generated from over 20,000 processes, including various event and error messages (Ren et al. 2021).

Experimental configuration

The experiment used two devices, a laptop host, installed with Windows 10 operating system, equipped with Intel(R) Core(TM) i7 processor, with python3.11 and pytorch development environment. A remote server, the server is equipped with 12th Gen intel® core™ i9-12900K×24 processor and Ubuntu 22.04.4 LTS operating system, with python3.11.7 and pytorch2.2.2 development environment. In the experiment, we set the initial learning rate to 0.001, the number of training rounds to 50, and the batch size to 1024.

Evaluation metrics

We evaluate the log anomaly detection method using precision, recall, and F1-score (Chen et al. 2022). These three indicators are calculated through TP, FP, TN, and FN, where TP means that abnormal logs are correctly judged as abnormal; FP means that abnormal logs are judged as normal; FN means that normal logs are judged as abnormal; TN means that normal logs are correctly judged as normal. The precision rate can be expressed as

Table 4 Similarity Performance under Different Location Similarity Weights

Location Similarity Weight(%)	Dataset		
	HDFS(%)	Hadoop(%)	BGL(%)
0	99.80	87.15	86.15
20	99.80	95.45	86.40
30	99.80	95.70	96.80
50	99.80	96.30	95.80
70	99.80	96.30	96.25
80	88.50	96.30	96.25
100	88.50	96.30	96.25

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (10)$$

which represents the percentage of anomalies that were detected correctly, out of reported anomalies. The definition of recall can be expressed as

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (11)$$

which shows the true anomalies that were correctly detected among all the true anomalies. The F1-score can be expressed as

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (12)$$

which represents the combination of precision and recall.

Experimental results

The effectiveness of log parse

We evaluate the effectiveness of LogLAA on three large datasets, and the parsing comparison is shown in Table 3. We compare LogLAA with the state-of-the-art (SOTA) model (Zhu et al. 2019) and other commonly adopted models in the LogPai benchmark. Other commonly adopted models are Logram (Dai et al. 2020), Spell (Du

Table 3 Comparison with state-of-the-art log parsers

	HDFS		Hadoop		BGL	
	Acc	F1-score	Acc	F1-score	Acc	F1-score
SOTA in LogPAI (Zhu et al. 2019)	1.000	0.999	0.947	0.998	0.953	0.999
Logram (Dai et al. 2020)	0.809	0.991	0.451	0.782	0.587	0.956
Spell (Du and Li 2016)	1.000	1.000	0.778	0.920	0.787	0.957
Brain (Yu et al. 2023)	0.997	0.999	0.949	0.998	0.970	0.999
LogLAA	0.998	1.000	0.963	0.997	0.963	1.000

and Li 2016) and Brain(Yu et al. 2023). Results show that LogLAA outperforms other models on each dataset, with a 0.8% accuracy improvement over the SOTA model.

Logs are typically presented in natural language to describe system events, with most content consisting of valid vocabulary defined in the dictionary. In addition, according to the experiments of Drain (He et al. 2017) et al., log messages with the same log event tend to have similar lengths. Therefore, our log parser uses a classification method based on word frequency and word length. This method treats log messages with similar word frequency and length as belonging to the same category. Fewer parameters and higher accuracy are achieved by combining variable substitution and combinatorial similarity clustering. Based on the results, we conclude that LogLAA shows excellent effectiveness and robustness across different log data.

Adaptability of combination similarity

In the log parser, we use combinatorial similarity clustering in log clustering to refine the clustering process. The

verification results are shown in Table 4. We evaluate the impact of different ratios of location similarity and jaccard similarity on clustering performance.

With high accuracy ensured, the best performance is achieved on all three datasets when Location Similarity accounts for 70% and Jaccard Similarity accounts for 30%.

Parser stability under different log numbers

To verify the impact of dataset size on the stability of the log parser, we select three datasets from the LogHub platform and evaluate performance by progressively increasing the log data. The dataset overview is shown in Table 2. Four proportions of 25%, 50%, 75%, and 100% are used for verification, and the verification results are shown in Table 5.

Through verification, the accuracy of the log parser proposed in this paper remains stable with different data volumes, consistently above 95%, and the F1 score exceeds 97% overall. This shows that the performance of the log parser is less affected by the log scale and has good stability.

Table 5 Evaluation Metrics under Different Anomaly Proportions

	HDFS		Hadoop		BGL	
	Acc	F1-score	Acc	F1-score	Acc	F1-score
25%	1.000	1.000	0.918	0.987	0.934	0.998
50%	1.000	1.000	0.937	0.989	0.966	0.999
75%	1.000	1.000	0.950	0.992	0.975	0.999
100%	0.998	1.000	0.963	0.997	0.963	1.000

Table 6 Comparison of anomaly detection methods

	HDFS			BGL		
	Precision	Recall	F1 score	Precision	Recall	F1 score
LogAnomaly (Meng et al. 2019)	0.886	0.917	0.901	0.957	1.000	0.978
LogRobust (Zhang et al. 2019)	0.741	0.994	0.849	0.996	0.998	0.997
Deeplog (Du et al. 2017)	0.856	0.999	0.922	0.962	1.000	0.980
FastLogAD (Lin et al. 2024)	0.876	0.922	0.899	0.811	0.810	0.810
LogLAA	0.916	0.960	0.937	0.999	0.999	0.999

Table 7 Ablation experiment on HDFS

Name	Attn	CNN	LSTM	SE	Precision	Recall	F1-score
ACLS	✓	✓	✓	✓	0.916	0.960	0.937
CLS	×	✓	✓	×	0.905	0.955	0.930
ALS	✓	×	×	✓	0.897	0.919	0.908
ACS	✓	✓	×	✓	0.855	0.945	0.898

The effectiveness of the anomaly detection model

Our goal is to evaluate the effectiveness of the CNN-LSTM model based on the Dual Attention Mechanism. We conducted experiments using the original HDFS and BGL datasets. We applied LogLAA's approach to extract log templates and used multiple methods (LogAnomaly, LogRobust, Deeplog, and FastLogAD) for anomaly detection. The detection results are shown in Table 6. LogAnomaly uses FT tree for log parsing, template2vec to extract semantic and grammatical information, and LSTM to detect sequence and quantitative anomalies (Meng et al. 2019). LogRobust extracts log templates through Drain (He et al. 2017) and uses LSTM based on attention mechanism for anomaly detection. DeepLog (Du et al. 2017) uses an LSTM-based model to model log key sequences for online anomaly detection and diagnosis (Cheng et al. 2025). FastLogAD (Lin et al. 2024) extracts log templates through Drain and builds a generator-discriminator model based on the ELECTRA architecture. The discriminator is trained using mask-guided pseudo-anomaly samples, thereby achieving efficient single-class discriminative log anomaly detection (Wang et al. 2022).

On the HDFS and BGL datasets, LogLAA achieves leading levels in Precision, Recall, and F1 scores. Compared to the average precision of other advanced methods, LogLAA improves anomaly detection precision by approximately 6%. By fusing semantic, sequence, and quantitative features, we address the challenges posed by log format instability. Additionally, we enhance the precision and robustness of anomaly detection by integrating the CNN-LSTM model with a dual attention mechanism.

Ablation experiment in HDFS

To evaluate the contribution of each component in the proposed log anomaly detection model, we designed an ablation experiment. This experiment systematically removes different submodules of the model. This approach helps us better understand the role and value of each part of the model, providing a solid foundation for subsequent optimization and improvement.

The original model consists of five key components: CNN module, LSTM network module, SE and self-attention mechanism (Attention). We remove each module separately and evaluate the performance of the model on the

same training set and test set. The verification results are shown in Table 7. We conducted ablation experiments on the HDFS dataset. By comparing the ablation model with the basic model, we find that the basic model performs best. This indicates that it is the most accurate in predicting logarithmic anomalies and demonstrates its effectiveness in minimizing false positives and false negatives.

This experiment emphasizes the importance of each module in model performance. Through comparative analysis, the baseline model has better performance in related evaluation indicators. It is verified that LSTM is a key component of the entire model and has a greater impact on the precision, recall and F1 score of the model. The attention mechanism and channel attention have less impact, but it does not affect their importance. The basic model without the attention mechanism and channel attention modules has good performance.

Validity of semantic features

LogAnomaly extracts semantic and quantitative features from logs and inputs them into the LSTM model for prediction. LogRobust extracts semantic and sequence features, feeding them into the bidirectional LSTM model based on the attention mechanism for prediction.

LogLAA uses a combination of semantic features, quantitative features, and sequence features. We evaluate validity of semantic features by removing their semantic features. The experimental results are shown in Table 8. We found that semantic features have a positive impact on both the precision and recall of predictions. This is because semantic features can better help understand the meaning of sentences and reduce the probability of false positives and false negatives.

Validity of anomaly explanations

To verify the effectiveness of anomaly explanation, we used the dataset published by Wang et al. (Wang et al. 2024), which consists of log-related replies tagged as "Apache" data on Stack Overflow. The dataset contains specific log and anomaly problem descriptions and solution strategies. By adjusting the dataset and using the anomaly explanation module in LogLAA, we obtained a result that includes log-human analysis-LogLAA analysis.

Table 8 Semantic feature stability on HDFS

	Has Semantic Feature			Without Semantic Feature		
	Precision	Recall	F1 score	Precision	Recall	F1 score
LogAnomaly (Meng et al. 2019)	0.886	0.917	0.901	0.876	0.824	0.850
LogRobust (Zhang et al. 2019)	0.741	0.994	0.849	0.737	0.972	0.838
LogLAA	0.916	0.960	0.937	0.909	0.955	0.932

Our evaluation method uses a weighted matching score to analyze feasibility. We use CAM (core action matching), SSS (solution similarity), and EAA (error attribution accuracy) to weight the indicators and get a total score to measure feasibility. CAM is an important similarity indicator that measures the degree of correct action matching. SSS is an indicator that measures the similarity between two solutions and is used to test the similarity between model proposals and manual annotations. EAA is an indicator that measures whether the system can accurately attribute errors to the correct causes in the error attribution task, which has a strong heuristic effect on whether the cause diagnosis is correct.

$$\text{CAM} = \frac{n_{\text{same}}}{n_{\text{total}}}. \quad (13)$$

$$\text{SSS} = \frac{E_{\text{Artificial}} \cdot E_{\text{Model}}}{\|E_{\text{Artificial}}\| \|E_{\text{Model}}\|}, \quad (14)$$

where $E_{\text{Artificial}}$ is the manually labeled feature vector of the abnormal cause, and E_{Model} is the feature vector generated by the model.

$$\text{EAA} = \begin{cases} 1 & \text{Exact match,} \\ 0.5 & \text{Partial match,} \\ 0 & \text{No match.} \end{cases} \quad (15)$$

We selected several datasets for testing and set the weights of the CAM, SSS, and EAA metrics to 0.6, 0.3, and 0.1. We took the weighted average of the weights for each metric to obtain a final score. Through experimental testing, the average CAM score was 0.8, the average SSS score was 0.54, and the average EAA score was 0.5. The weighted total score was 0.7, which is a relatively good level.

CAM core action matching can provide good feedback on whether the correct treatment measures have been taken. To better extract effective vocabulary for key steps, we use ChatGPT to identify and analyze feature words, effectively obtaining the CAM score. The CAM score of 0.8 shows LogLAA can well identify and execute key steps similar to the manual solution, and performs well in the core logic of the solution. SSS focuses on semantic comparison. LLMs offer broader coverage when analyzing problems, but their solutions are less targeted. The EAA error attribution accuracy score is 0.5, indicating that the manual solution and the model solution partially match in terms of error attribution. The model can identify some of the root causes in the manual solution, but not completely consistent. It may be that the model solution considers more factors, resulting in a wider attribution range for the model, which may contain more potential problems. The overall score of the framework is 0.7, which is at an upper-middle level, indicating that the

framework can maintain a relatively good level when performing anomaly analysis.

Conclusion and future work

When distributed systems are running, they generate interleaved logs, which complicate the log analysis process. To achieve more robust and accurate anomaly detection and analysis, we propose a model called LogLAA. This model improves the stability of anomaly detection by building a log parser based on word frequency, word length and combined with similarity prefix tree clustering. We significantly improve the precision of anomaly detection by introducing a combination of semantic embedding, sequence embedding, and count embedding, combined with a CNN-LSTM model based on a dual attention mechanism. We create a relational mapping, extract abnormal logs in real time, and perform online anomaly analysis using a LLM. Experimental results show that our log parser outperforms other log parsers. Compared with other advanced methods, LogLAA improves the precision of anomaly detection and is able to perform anomaly cause analysis on real data sets, providing a new perspective for anomaly detection and analysis in distributed systems. Through performance evaluation, our framework can also achieve a good level in anomaly analysis. Although the log parsing technology in this paper has improved parsing efficiency, the effectiveness of log parsing may decrease as the complexity of the log structure increases. In addition, as the number of components in a distributed system increases, the efficiency and accuracy of anomaly detection may be affected. Therefore, it becomes very valuable to explore alternative ways to tune log parsing strategies and anomaly localization to improve the overall performance.

Acknowledgements

We would like to thank the National Key Research and Development Program of China under Grant No.2023YFB3107601.

Author contributions

YG: Led the overall project, provided guidance on methodology design, supervised all stages of the work, and made substantial revisions to the manuscript. TL: Conceived the core idea, implemented the proposed methods, conducted the experiments, and drafted the manuscript. HK: Writing review and editing and data processing. TJ: Writing review and editing and data processing. XL: Supervised the overall project, offered strategic guidance and direction.

Funding

This work was supported by the National Key Research and Development Program of China under grant No.2023YFB3107601.

Data Availability

All log datasets are available at <https://github.com/logpai/loghub>.

Declarations

Consent for publication

All authors have read and approved the final manuscript and consent to its publication.

Competing interests

The authors declare that they have no conflict of interest.

Received: 4 November 2025 Accepted: 6 March 2026

Published online: 20 March 2026

References

- Chen X, He B, Sun L (2022) Groupwise query performance prediction with bert. In: European Conference on Information Retrieval, Springer, pp 64–74
- Chen Y, Xie H, Ma M, et al (2024) Automatic root cause analysis via large language models for cloud incidents. In: Proceedings of the Nineteenth European Conference on Computer Systems, pp 674–688
- Cheng H, Xu D, Yuan S (2025) Backdoor attack against log anomaly detection models. Companion Proceedings of the ACM on Web Conference 2025:915–918
- Chnib M, Gabisi W (2023) Detection of anomalies in the hdfs dataset. 2023 IEEE/ACIS 21st International Conference on Software Engineering Research. IEEE, Management and Applications (SERA), pp 243–250
- Dai H, Li H, Chen CS et al (2020) Logram: Efficient log parsing using n -gram dictionaries. *IEEE Trans Software Eng* 48(3):879–892
- Devlin J, Chang MW, Lee K, et al (2019) Bert: Pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers), pp 4171–4186
- Du M, Li F (2016) Spell: Streaming parsing of system event logs. In: 2016 IEEE 16th International Conference on Data Mining (ICDM), IEEE, pp 859–864
- Du M, Li F, Zheng G, et al (2017) Deeplog: Anomaly detection and diagnosis from system logs through deep learning. In: Proceedings of the 2017 ACM SIGSAC conference on computer and communications security, pp 1285–1298
- Duan C, Jia T, Yang Y et al (2025) Eagerlog: Active learning enhanced retrieval augmented generation for log-based anomaly detection. *ICASSP 2025–2025 IEEE International Conference on Acoustics, IEEE, Speech and Signal Processing (ICASSP)*, pp 1–5
- Guan W, Cao J, Qian S, et al (2024) Logllm: Log-based anomaly detection using large language models. *arXiv preprint arXiv:2411.08561*
- He M, Jia T, Duan C et al (2024) Llmelog: An approach for anomaly detection based on llm-enriched log events. 2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE). IEEE, Tsukuba, Japan, pp 132–143
- He M, Jia T, Duan C, et al (2025) Weakly-supervised log-based anomaly detection with inexact labels via multi-instance learning. In: 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), IEEE Computer Society, pp 2918–2930
- He P, Zhu J, Zheng Z, et al (2017) Drain: An online log parsing approach with fixed depth tree. In: 2017 IEEE international conference on web services (ICWS), IEEE, pp 33–40
- Jiang ZM, Hassan AE, Flora P, et al (2008) Abstracting execution logs to execution events for enterprise applications. In: 2008 The Eighth International Conference on Quality Software, IEEE, pp 181–186
- Kenter T, Borisov A, de Rijke M (2016) Siamese CBOW: Optimizing word embeddings for sentence representations. In: Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). Association for Computational Linguistics, Berlin, Germany, pp 941–951, <https://doi.org/10.18653/v1/P16-1089>, <https://aclanthology.org/P16-1089/>
- Kim C, Jang M, Seo S et al (2021) Intrusion detection based on sequential information preserving log embedding methods and anomaly detection algorithms. *IEEE Access* 9:58088–58101
- Lan Z, Chen M, Goodman S, et al (2019) Albert: A lite bert for self-supervised learning of language representations. In: International Conference on Learning Representations
- Landauer M, Onder S, Skopik F et al (2023) Deep learning for anomaly detection in log data: A survey. *Machine Learning with Applications* 12:100470
- Le VH, Zhang H (2022) Log-based anomaly detection with deep learning: How far are we? In: Proceedings of the 44th international conference on software engineering, pp 1356–1367
- Lin Y, Deng H, Li X (2024) Fastlogad: Log anomaly detection with mask-guided pseudo anomaly generation and discrimination. *arXiv preprint arXiv:2404.08750*
- Liu Y, Tao S, Meng W, et al (2024) Logprompt: Prompt engineering towards zero-shot and interpretable log analysis. In: Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings, pp 364–365
- Liu Y, Ji Y, Tao S, et al (2025) Loglm: From task-based to instruction-based automated log analysis. In: 2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), pp 401–412, <https://doi.org/10.1109/ICSE-SEIP66354.2025.00041>
- Meng W, Liu Y, Zhu Y, et al (2019) Loganomaly: Unsupervised detection of sequential and quantitative anomalies in unstructured logs. In: *IJCAI*, pp 4739–4745
- Novaes MP, Carvalho LF, Lloret J et al (2020) Long short-term memory and fuzzy logic for anomaly detection and mitigation in software-defined network environment. *Ieee Access* 8:83765–83781
- Ren R, Cheng JC, Shi H et al (2021) Failure characterization based on lstm networks for bluegene/l system logs. *Intelligent Computing and Block Chain: FICC 2020*. Springer Singapore, Singapore, pp 123–133
- Sun Y, Gao Y, Li X (2023) OptimizeLog: Log anomaly detection and localization based on optimized log parsing in distributed systems. 2023 9th International Conference on Computer and Communications (ICCC). IEEE, Chengdu, China, pp 2144–2148
- Vaarandi R, Pihelgas M (2015) Logcluster-a data clustering and pattern mining algorithm for event logs. In: 2015 11th International conference on network and service management (CNSM), IEEE, pp 1–7
- Wang J, Chu G, Wang J, et al (2024) Logexpert: Log-based recommended resolutions generation using large language model. In: Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, pp 42–46
- Wang Z, Tian J, Fang H et al (2022) Lightlog: A lightweight temporal convolutional network for log anomaly detection on the edge. *Comput Netw* 203:108616
- Xiao Y, Le VH, Zhang H (2024) free: Towards more practical log parsing with large language models. In: Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, pp 153–165
- Xu W, Huang L, Fox A, et al (2009) Largescale system problem detection by mining console logs. In: Proceedings of SOSP, pp 1–17
- Ying S, Wang B, Wang L et al (2021) An improved knn-based efficient log anomaly detection method with automatically labeled samples. *ACM Transactions on Knowledge Discovery from Data (TKDD)* 15(3):1–22
- Yu S, He P, Chen N et al (2023) Brain: Log parsing with bidirectional parallel tree. *IEEE Trans Serv Comput* 16(5):3224–3237
- Zhang X, Xu Y, Lin Q, et al (2019) Robust log-based anomaly detection on unstable log data. In: Proceedings of the 2019 27th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering, pp 807–817
- Zhao X, Jia T, He M, et al (2025) From few-label to zero-label: An approach for cross-system log-based anomaly detection with meta-learning. In: Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering, pp 661–665
- Zhou J, Qian Y, Zou Q et al (2022) Deepsyslog: Deep anomaly detection on syslog using sentence embedding and metadata. *IEEE Trans Inf Forensics Secur* 17:3051–3061
- Zhu J, He S, Liu J, et al (2019) Tools and benchmarks for automated log parsing. In: 2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP), IEEE, pp 121–130
- Zhu J, He S, He P, et al (2023) Loghub: A large collection of system log datasets for ai-driven log analytics. In: 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), IEEE, pp 355–366
- Zuo C, Qian J, Feng S et al (2022) Deep learning in optical metrology: a review. *Light Sci Appl* 11(1):39

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.